

Pázmány Péter Katolikus Egyetem
Információs Technológiai és Bionikai Kar

Önálló laboratórium

2025/2026/2 félév

**Multithreadelt C++ alapú lokális cache szerver
teljesítményvizsgálata és optimalizációs lehetőségeinek
elemzése**

Hallgató:

Juhász Bence

Szak: IANI-MI – Mérnökinformatikus

BSc

Témavezető:

Dr. Feldhoffer Gergely

Budapest, 2026

Tartalomjegyzék

1. Feladat ismertetése	2
2. Bevezetés	3
2.1. A téma motivációja	3
2.2. Elméleti háttér	3
2.3. A vizsgált hipotézisek	4
3. A konkrét munka ismertetése	4
3.1. Mérési környezet és módszertan	5
3.2. A szerver implementációja	5
3.3. H1: Telítési pont a kliensszám függvényében	6
3.3.1. Hipotézis és felállítása	6
3.3.2. Eredmények	7
3.3.3. Diskusszió	8
3.4. H2: Worker pool méretének optimuma	9
3.4.1. Hipotézis és felállítása	9
3.4.2. Eredmények	10
3.4.3. Kontrollmérés 2 magos pinninggel	11
3.4.4. Diskusszió	11
3.5. H3: Task queue méretének hatása	12
3.5.1. Hipotézis és felállítása	12
3.5.2. Eredmények	13
3.5.3. Diskusszió	15
3.6. H4: Task típusok skálázódási profiljának összehasonlítása	16
3.6.1. Hipotézis és felállítása	16
3.6.2. Eredmények	17
3.6.3. Diskusszió	18
3.7. Összegző megfigyelések	18
4. Összefoglalás	18
4.1. A hipotézisek áttekintése	18

1. Feladat ismertetése

A dolgozat célja egy C++ nyelven implementált, több szálon működő lokális cache szerver tervezése és vizsgálata, különös tekintettel a teljesítményoptimalizálási lehetőségekre és azok mérési módszertanára. A rendszer egy párhuzamos feldolgozást alkalmazó szerver architektúrát valósít meg, amely klienskéréseket fogad, azokat worker szálak között osztja szét, és a válasz előállítása során számításigényes feladatokat hajt végre. A számításigényes műveletek célja a CPU-terhelés kontrollált növelése, hogy a különböző konkurens megoldások és cache-stratégiák hatása mérhetővé váljon.

A cache szerver működését három különböző típusú algoritmussal modelleztem, amelyek célja az, hogy hűen reprezentálják a CPU-bound, IO-bound és hálózaton elérhető erőforrástól függő feladatok viselkedését egy párhuzamos végrehajtási környezetben. Ezek a feladatok végrehajtásra kerülnek minden egyes klienskérés hatására, majd a kliensek választ kapnak ezek eredményéről hálózaton keresztül.

A CPU-bound feladatok nagy mértékben használják ki a rendelkezésükre álló processzoridőt, amíg az IO-bound feladatok ezzel ellentétben a lemezmeghajtó sávszélességét igyekeznek kihasználni, ami során az adatátvitel jellege miatt sűrűn várakozásra kényszerülnek, ezért a processzoridőt kevésbé használják ki. A hálózaton elérhető erőforrástól függő feladatok ezutóbbiakhoz hasonlóak, azzal a különbséggel, hogy azok esetében nem a lemezzől, hanem a hálózatról érkező adatokra kell várni az olvasás során adott időközönként. Ezen feladatok számára a véges processzoridő és a lemez elérésének késleltetése korlátként jelenik meg, ezáltal alkalmasak arra, hogy rajtuk keresztül vizsgáljam meg, hogy a különböző módokon végzett párhuzamos végrehajtásuk hogyan hat ezekre a korlátaikra.

A méréseim során arra kerestem a választ, hogy az általam tervezett szerverarchitektúra szabad paraméterei hogyan hangolhatóak annak érdekében, hogy ezek a feladatok a lehető legjobban kihasználhassák a tesztrendszerem erőforrásait, illetve hogy ennek a vizsgálata során milyen eltérések mutatkoznak a különböző szűk keresztmetszettel rendelkező algoritmusok között.

A feladataimhoz hozzátárgya azt tapasztaltam, hogy sok lehetőség állt rendelkezésemre, mivel mind a tesztfeladatok, a szerverarchitektúra és a mérési környezet kialakításában szabadságom volt. Annak érdekében, hogy a munkám jól definiált struktúrát kapjon, öt részre bontottam: Először magát a szerverprogramot implementáltam, majd a korábbi ismereteimre hagyatkozva négy hipotézist állítottam fel ennek a működéséről. A későbbiekben ezeknek a hipotéziseknek az igazságtartalmát vizsgáltam mérésekkel és azok értelmezésével.

A dolgozatban a párhuzamos feldolgozási műveleteket végző szálakat reprezentáló objektumokra workerként, összességükre worker poolként, a kérések kiszolgálásának idejére késleltetésként, latencyként, ennek adott legkedvezőtlenebb százalékára pN% tail latencyként, a beérkező kéréseket

a feldolgozás megkezdése előtt tároló, FIFO sor jellegű adatszerkezetekre queue-ként vagy task queue-ként, az egységnyi idő alatt kiszolgált kérésekre throughputként, a terhelést modellező implementációkra taskként hivatkozok.

2. Bevezetés

2.1. A téma motivációja

Az Önálló laboratóriumi feladatom során az volt a célom, hogy megismerjem, milyen szempontok mentén lehet tervezni egy több szálon működő C++ szerverkomponenst, valamint hogy az ilyen jellegű működéshez milyen eszközöket biztosít ez a programnyelv. Kíváncsi voltam továbbá arra is, hogy az általam készítendő terhelést modellező programok hogyan skálázódnak a processzormagokon a szimulált kliensek számával, ugyanis nem üzemeltettem még olyan rendszert, ahol a szűk keresztmetszet a kliensek számával vált volna jelentőssé. Mindezt annak érdekében szerettem volna megvalósítani, hogy ezáltal olyan tapasztalatokat szerezzek, amelyeket később a gyakorlatban is hasznosíthatok, amikor majd egy nagyobb rendszer teljesítményének optimalizálásán fogok dolgozni például egy ipari környezetben.

2.2. Elméleti háttér

A sorbanállás-elmélet (queueing theory) olyan matematikai eszközrendszert nyújt, amellyel várakozási sorok viselkedése modellezhető. Egy ilyen modell alapvető összefüggése Little's törvénye [1], amely kimondja, hogy egy stabil rendszerben az egyidejűleg jelen lévő kérések átlagos száma (L) egyenlő az érkezési rátának (λ) és az átlagos rendszerben töltött időnek (W) a szorzatával:

$$L = \lambda \cdot W \tag{1}$$

Ez az összefüggés független az időbeli eloszlástól, amennyiben a rendszer stabil állapotban van. A méréseim során CPU- és IO-bound taskokat, valamint egy külső, hálózaton keresztül elérhető erőforrásra tmáaszkodó taskot vizsgáltam, amelyek kiszolgálási idő eloszlása egymástól jelentősen eltér. Viszont az időbeli eloszlástól való függetlenség miatt mindhármat érdemes a Little's Law szemszögéből vizsgálni.

A vizsgált szerver felépítése megfeleltethető az M/M/c modellnek [2], amelyben a kérések Poisson-folyamat szerint érkeznek (λ rátával), a kiszolgálási idők exponenciálisan oszlanak el, és c párhuzamos kiszolgáló (worker) áll rendelkezésre. Ha $\rho = \lambda/(c\mu)$ a kihasználtsági tényező (ahol μ az egy worker kiszolgálási rátája), akkor $\rho < 1$ esetén a rendszer stabil, és a várakozási sor véges marad. Ha $\rho \geq 1$, a sor korlátlanul növekszik, a válaszidő divergál: ez megfelel a mérésekben megfigyelt telítési jelenségnek.

A valós rendszer több ponton eltér az ideális M/M/c modelltől. A kiszolgálási idők nem exponenciálisak: a CPU-bound task determinisztikusan fut, az IO-bound task eloszlását a fájlrend-

szer viselkedése, a network task eloszlását pedig az upstream szerver válaszája befolyásolja. Emellett a worker pool mérete véges, és a task queue kapacitása korlátozott, ami kérések elutasításához vezet. Ezek az eltérések magyarázhatják, ha a mérési eredmények nem pontosan a modell előrejelzései szerint alakulnak, és a hipotézisek értelmezésénél egyaránt figyelembe kell venni őket.

2.3. A vizsgált hipotézisek

- **H1:** Telítési pont létezése a kliensszám függvényében
- **H2:** Worker pool méretének hatása a teljesítményre
- **H3:** Task queue méretének hatása a latencyre és a reject rátára
- **H4:** Skálázódási profil task típusonként

3. A konkrét munka ismertetése

Az Önálló laboratóriumi munkám során először elkészítettem a C++ szerver program implementációját. Ennek kezdetben az Internetről lehető legtöbb forrást felhasználva álltam neki, beleértve google keresési találatokat a stackexchange.com fórumról és a generatív nagy nyelvi modellek kimeneteit is. Amikor a tesztjeim során elkezdett működni egy így megismert koncepció, ellenőriztem, hogy a cppreference.com, a C++ szabvány dokumentációja szerint valóban arra a célra való-e az általam újonnan megismert komponens, mint amire éppen használok. Ezt követően iteratívan egyeztettem a témavezetőmmel a szoftver állapotáról, hogy pontosítást kapjak a megközelítésemben.

Segítségemre voltak továbbá a párhuzamos feladatokat megvalósító kód implementálása során azok az ismeretek, amelyeket korábban a Java programozás című tárgyon sajátítottam el.

Az implementáció során fény derült arra, hogy a szerverprogramnak a felépítéséből fakadóan két olyan szabadon hangolható paramétere van, amelyeknek a teljesítményre gyakorolt hatását érdemes megvizsgálni. Az egyik ilyen a worker pool mérete, a másik pedig a task queue mérete. A később felállított hipotéziseim megfogalmazása és a mérések során lényegében ezeknek a hatásával foglalkoztam.

A mérések során a wrk nevű HTTP benchmarking eszközt használtam, amelynek segítségével különböző számú klienst tudtam szimulálni, és így megvizsgálni a szerver teljesítményét a kliensszám függvényében. Ez az eszköz arra is alkalmas, hogy a késleltetési idő különböző statisztikai jellemzőit, például a p99 értékét is mérje, ami a dolgozatban fontos szerepet játszik a teljesítmény elemzésében. Egy ilyen érték például azt mutatja meg, hogy a kérések adott százaléka milyen késleltetési időn belül teljesül, ami egy fontos mérőszám a szerver válaszájának értékeléséhez, ugyanis ezzel kifejezhető, hogy a kevésbé ideális esetekben hogyan viselkedik a program. A mérési eredmények feldolgozásához Python programokat írtam, amelyek segítségével rendszereztem és elemeztem az adatokat, valamint előállítottam a dolgozatban szereplő ábrákat.

3.1. Mérési környezet és módszertan

A méréseim során egy HP ZBook 17 G4 típusú laptopot használtam Intel Core i7-7820HQ processzorral, ami négy fizikai processzormaggal rendelkezik, valamint az Intel ún. Simultaneous Multi-Threading (SMT) technológiájával nyolc szálon tesz lehetővé párhuzamos műveletvégzést.

Az operációs rendszer kiválasztásánál a reprodukálhatóság érdekében egy olyan linux-disztribúcióra, az Arch Linuxra esett a választásom, amely arra törekszik, hogy minél inkább kövesse az azt felépítő komponensek eredeti működését, konfigurációját, kevés disztribúcióspecifikus módosítást tartalmazzon.

A méréseimet a Linux kernel 7.0.3 verzióját használva futtattam, a fájlrendszer btrfs volt.

A HTTP kérések generálásához a wrk program 4.2.0-3-es verzióját telepítettem az Arch Linux csomagtárolóból.

Az általam írt szerver és a wrk program terhelését a processzormagok között a taskset nevű programmal osztottam el, amit a util-linux nevű csomag 2.42-1-es verziójából szereztem be. A nyolc logikai mag közül az első négyet, 0-3-ig a szerver számára osztottam ki, az utolsó kettőt, 6-7 azonosítóval pedig a wrk mérőeszköz számára. Két processzormagot, 4-5 azonosítóval nem érintett a mérésem. Ezt a felosztást annak érdekében valószínűsítem meg, hogy a szerver mellett futó mérőeszköz ne okozhasson zajt a saját működésével a méréseimben azáltal, hogy kontextusváltást igényelne azokon a processzormagokon, amelyeken a vizsgált program dolgozik. Habár igyekeztem minimalizálni a háttérben futó többi program mennyiségét, az operációs rendszer érdemben való használhatóságához elkerülhetetlen, hogy néhány ilyen jellegű szoftver jelen legyen. Ezek számára azonban az operációs rendszer ki tudta osztani a mérés által nem befolyásolt maradék 4-es és 5-ös processzormagot. A mérés közben, hogy nyomon tudjam követni annak állapotát, a sway nevű ablakkezelő program futott, amelynek rendszerigénye alacsonyabb, mint például egy GNOME vagy KDE környezeté.

A szervert a GNU Compiler Collection (GCC) segítségével fordítottam, a 16.1.1-es verziójával.

A mérési adatokat Python programok segítségével állítottam elő és rendszereztem, valamint dolgoztam fel.

3.2. A szerver implementációja

A szerverprogram a következő fő komponensekből áll:

- **Main szál:** Ez a komponens felelős a konfigurációs paraméterek feldolgozásáért, a TCP socket létrehozásáért és a bejövő kapcsolatok fogadásáért. Amikor egy új kliens csatlakozik, a main szál egy új taskot hoz létre, amely tartalmazza a kliens kérését, és ezt a taskot elhelyezi a worker pool által használt task queue-ba.
- **Worker pool:** Ez egy fix méretű szálcsoport, amely a main szál által létrehozott taskokat dolgozza fel. A worker szálak egy közös task queue-t használnak, ahonnan felveszik a feldolgozásra váró taskokat. A worker pool mérete konfigurálható, és ez az egyik vizsgált

paraméter a dolgozatban.

- **Task típusok:** A szerver három különböző típusú taskot képes feldolgozni: CpuWorker, IoWorker és egy default Worker, amely egy egyszerű network proxyként működik. A CpuWorker típusú taskok számításigényes műveleteket hajtanak végre, míg az IoWorker típusú taskok a lemezhez kapcsolódó műveleteket végeznek. A default Worker egyszerűen továbbítja a kéréseket egy upstream szerver felé, és visszaküldi a válaszokat a kliensnek.

3.3. H1: Telítési pont a kliensszám függvényében

3.3.1. Hipotézis és felállítása

A szerver válaszideje a párhuzamos kliensek számának függvényében nem-lineáris karakterisztikát mutat: létezik egy kliensszám-küszöb (knee point), amelyen túl a válaszidő meredeken növekszik, miközben a throughput telítődik vagy csökken.

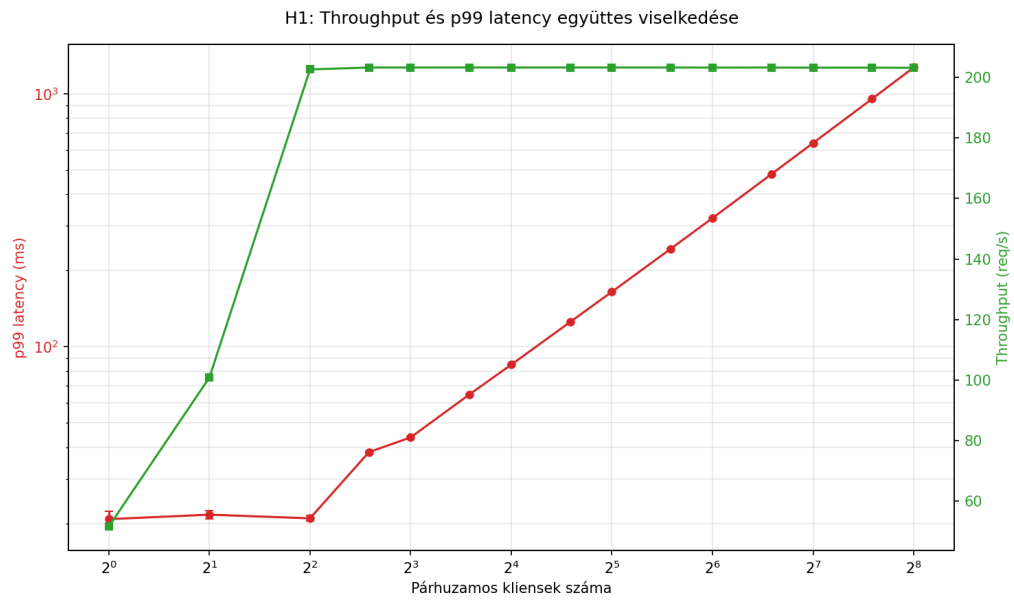
A mérési elrendezés során a worker pool mérete fixen négy, minden worker egy független processzormagon dolgozhat. A task queue méretét nagyra, 2048-ra választottam, hogy biztosan ne keletkezzen olyan eset, amikor megtelik, és emiatt a programnak el kell utasítania a bejövő kérések kiszolgálását. A változó paraméter a mérés során a kliensek száma, rendre 1, 2, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256 értékeken, hogy széles tartományban vizsgálhassam a szerver viselkedését a kliensszám függvényében, és így pontosan beazonosíthassam azt a pontot, ahol telítődési jelenség jelentkezik. A teljes mérést ötször futtattam, hogy átlagolni tudjam a kapott eredményeket minden beállítás mellett. Egy beállítás 60 másodpercig tartott, hogy elegendő időt biztosítson a stabil mérési eredmények eléréséhez, különösen a tail latency tekintetében. Köztük öt másodperc szünetet hagytam, hogy a rendszer vissza tudjon állni a kiindulási állapotba, és ne legyen hatással a következő mérésre.

Előttük egy tíz másodperces "bemelegítési" futtatást csináltam, hogy még a mérés előtt hasonló állapotba kerülhessen a rendszer, mint futás közben. Ez alatt nem rögzítettem adatokat. Ezt követően két másodperc várakozási időt állítottam be, hogy a "bemelegítési" szakaszban futtatott terhelés közvetlenül ne befolyásolja az utána kezdődő tényleges mérést.

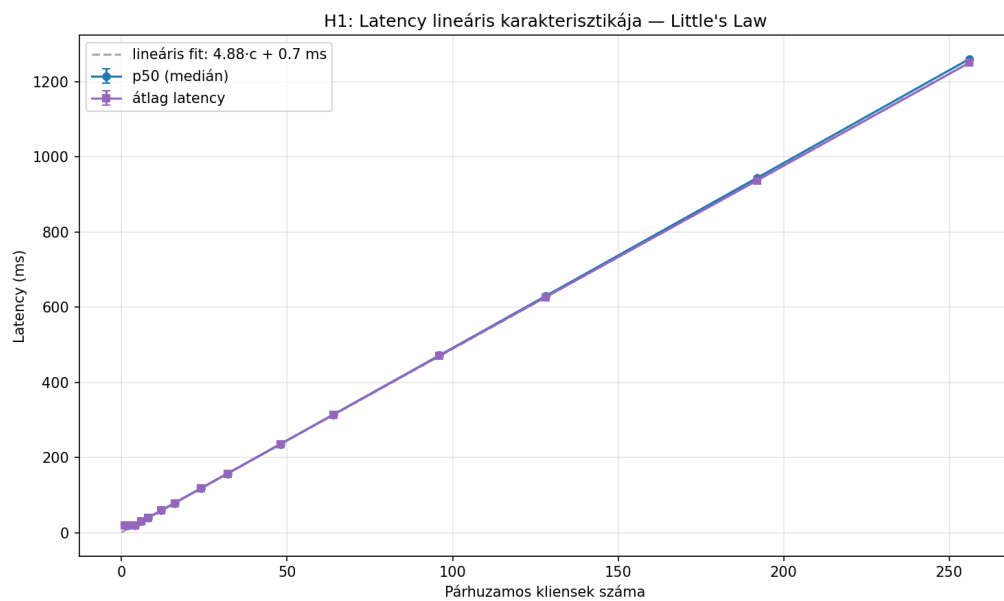
A nyolc logikai processzormag közül az első négyet, 0-3-ig a szerver számára osztottam ki, az utolsó kettőt, 6-7 azonosítóval pedig a wrk mérőeszköz számára. Két processzormagot, 4-5 azonosítóval nem érintett a mérésem.

H1 igazolt, ha a latency-kliensszám görbén beazonosítható egy pont, ahol a görbe meredeksége legalább 5x-ösére nő egy szűk tartományon belül, és a throughput ezen a ponton $\pm 10\%$ -on belül stagnál. Várt eredmény: "Hockey stick" görbe; a pont a worker pool méretének közelében jelenik meg CPU-bound task esetén.

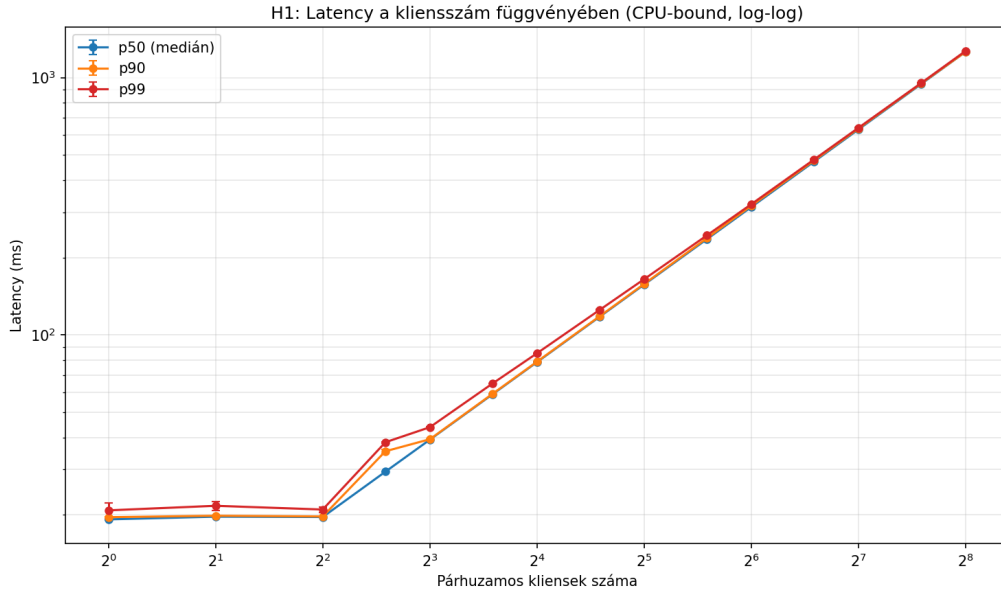
3.3.2. Eredmények



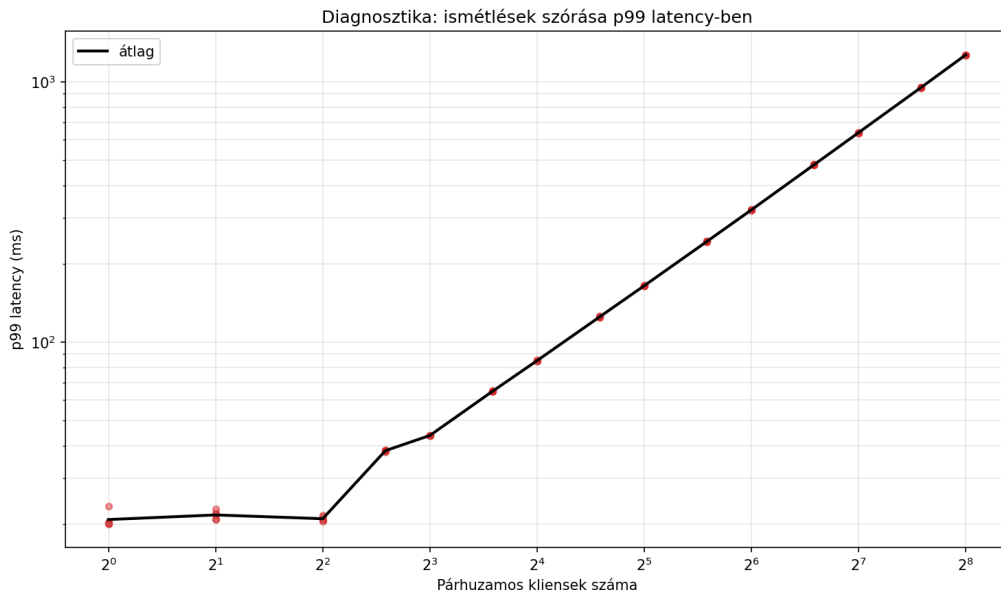
1. ábra. H1: Throughput és p99 latency együttes viselkedése a kliensszám függvényében.



2. ábra. H1: Latency lineáris karakterisztikája — Little's Law szerinti illeszkedés.



3. ábra. H1: TODO szöveg



4. ábra. H1: TODO szöveg

3.3.3. Diskusszió

A mérési eredmények a hipotézist egyértelműen igazolják. A throughput $c = 4$ párhuzamos kliensnél eléri a 203 req/s telítettségi szintet, és ezt követően egészen $c = 256$ -ig stabilan tartja ezt az értéket (1. ábra). A telítési pont pontosan a worker pool méreténél jelenik meg, ami összhangban van az M/M/c modell előrejelzésével: amikor a párhuzamos kliensek száma eléri a workerek számát, a kihasználtsági tényező $\rho \approx 1$ -re növekszik, és a rendszer telítődik.

A latency ezzel párhuzamosan $c = 4$ -ig közel konstans (20 ms), majd ezt követően folyamatosan és meredeken növekszik (3. ábra). Figyelemre méltó, hogy a p50, p90 és p99 percentilis görbék

$c = 4$ felett szinte teljesen egybeesnek, ami azt jelzi, hogy a telítés után nem csupán a legrosszabb esetű válaszidők romlanak, hanem a válaszidők teljes eloszlása eltolódik. Ez arra utal, hogy a várakozási sor dominál a válaszidőben, nem pedig az egyedi kérések feldolgozási idejének szórása.

A 2. ábra a latency lineáris karakterisztikáját mutatja be. A mért p50 és átlag latency értékekre illesztett lineáris függvény $W = 4,88 \cdot c + 0,7$ ms alakú, és az illeszkedés szinte tökéletes. Ez Little's törvényének közvetlen igazolása: mivel a throughput telítés után konstans (λ fix), W lineárisan arányos a rendszerben lévő kérések számával ($L = c$).

A diagnosztikai ábra (4. ábra) tanúsága szerint az öt ismételtes szórása minden kliensszámánál elhanyagolható, ami a mérések jó reprodukálhatóságát igazolja.

3.4. H2: Worker pool méretének optimuma

3.4.1. Hipotézis és felállítása

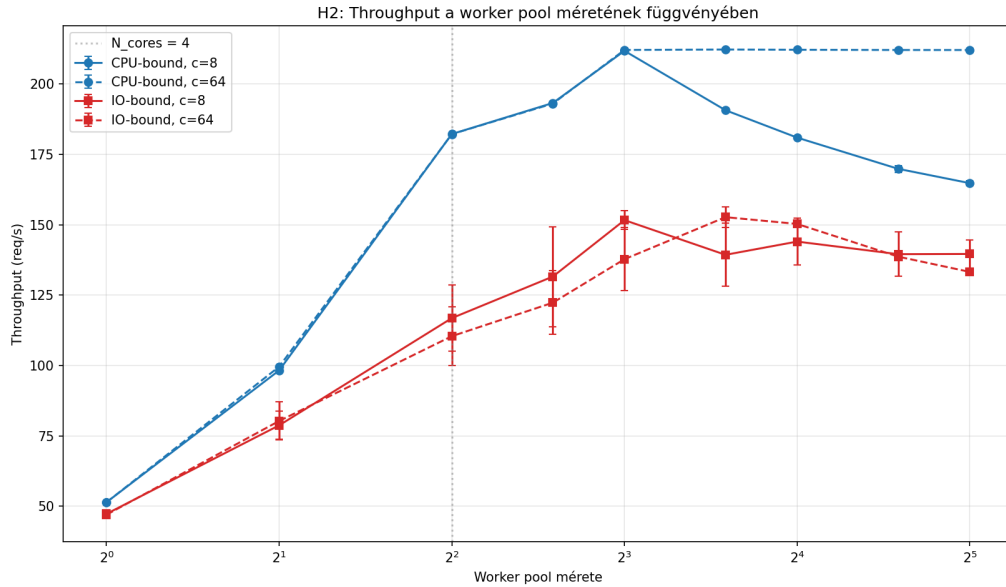
Minden feladattípushoz, amit a worker pool végrehajt, létezik egy hozzá tartozó optimális worker pool méret. Amikor a processzor a szűk keresztmetszet, akkor ez a méret körülbelül egyenlő a rendelkezésre álló processzormagok számával, +/- 10míg amikor az IO műveletek a szűk keresztmetszet, akkor ez a méret jelentősen nagyobb, akár 4-10-szerese is lehet a processzormagok számának.

A mérés célja annak megállapítása, hogy hogyan lehet kihasználni minél jobban a rendelkezésre álló erőforrásokat, úgy, hogy közben ne keletkezzen olyan overhead, amely a kontextusváltások miatt rontja a válaszidőt.

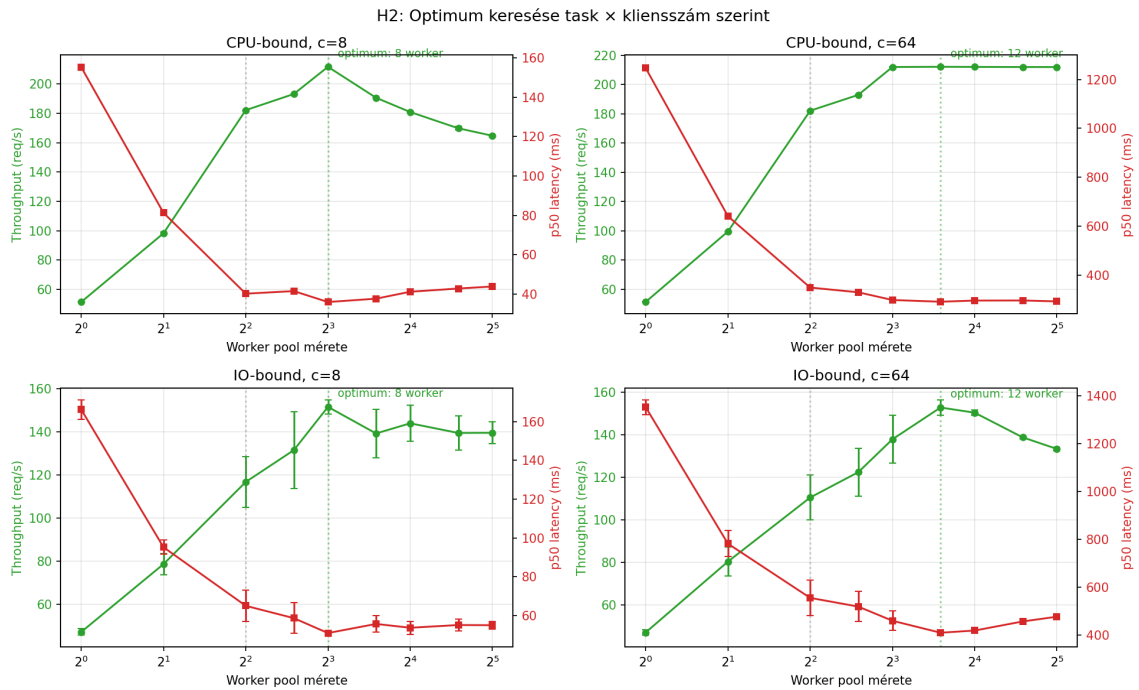
A mérés elrendezése során rögzített kliensszámot választottam, hogy ez ne befolyásolja a válaszidőt, tisztán a worker pool méretének hatása jelentkezzen. Azonban a két típusú, IO és CPU-bound feladatok eltérő jellegéből adódóan úgy döntöttem, hogy két különböző kliensszámmal is elvégzem a mérést mindkét esetben, mert különböző mértékben kezdenek el telítődni az erőforrások ezek esetén. A két választott kliensszámom 8 és 64 lett, ezeknél már mindkét esetben meg kell, hogy jelenjen a telítődés H1 eredményei szerint. A queue méretét ismét nagyra, 2048-ra választottam, hogy ez ne legyen szűk keresztmetszet a mérés során. A worker pool méretét rendre 1, 2, 4, 6, 8, 12, 16, 24, 32 értékeken iteráltam a mérés során. A teljes mérést ötször futtattam, hogy átlagolni tudjam a kapott eredményeket minden beállítás mellett. Egy beállítás 60 másodpercig futott, hogy elegendő időt biztosítson a stabil mérési eredmények eléréséhez, különösen a tail latency tekintetében. Köztük öt másodperc szünetet hagytam, hogy a rendszer vissza tudjon állni a kiindulási állapotba, és ne legyen hatással a következő mérésre. Előttük egy tíz másodperces "bemelegítési" futtatást csináltam, hogy még a mérés előtt hasonló állapotba kerülhessen a rendszer, mint futás közben. Ez alatt nem rögzítettem adatokat. Ezt követően két másodperc várakozási időt állítottam be, hogy a "bemelegítési" szakaszban futtatott terhelés közvetlenül ne befolyásolja az utána kezdődő tényleges mérést. A nyolc logikai processzormag közül az első négyet, 0-3-ig a szerver számára osztottam ki, az utolsó kettőt, 6-7 azonosítóval pedig a wrk mérőeszköz számára. Két processzormagot, 4-5 azonosítóval nem érintett a mérésem.

H2 igazolt, ha a CPU-bound taskok esetén a worker pool méretének optimuma a processzormagok számának közelében van (± 10 és efölött válaszdő romlás mérhető, míg az IO-bound taskok esetén ez az optimum jelentősen nagyobb, legalább kétszerese a processzormagok számának.

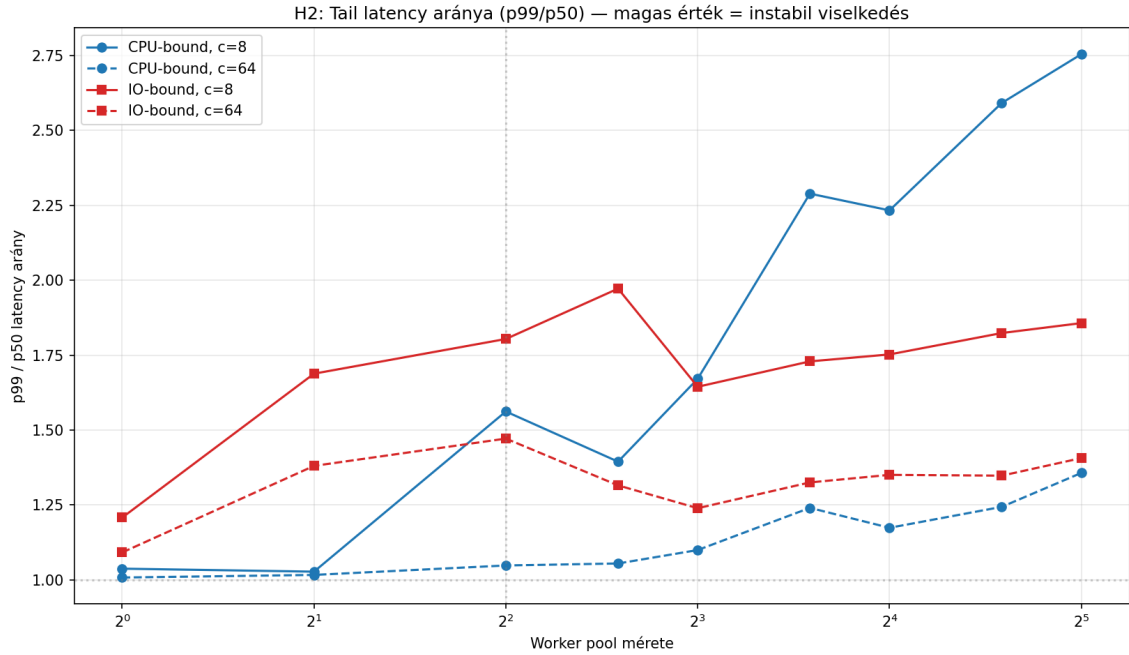
3.4.2. Eredmények



5. ábra. H2: Throughput a worker pool méretének függvényében task típusonként.



6. ábra. H2: Optimum keresése task x kliensszám szerint, throughput és latency együtt.



7. ábra. H2: TODO szöveg

3.4.3. Kontrollmérés 2 magos pinninggel

.

3.4.4. Diszkusszió

A mérési eredmények a hipotézist részlegesen cáfolják. A hipotézis szerint azt vártam, hogy CPU-bound taskok esetén az optimális worker pool mérete a fizikai processzormagok számának közelében (≈ 4) lesz, az IO-bound taskok esetén pedig ennek legalább kétszerese. Ezzel szemben mindkét task típusnál az optimum 8 workernél jelent meg $c = 8$ kliensszám mellett, és 12 workernél $c = 64$ esetén: mindkettő meghaladja a fizikai magok számát (5. és 6. ábra).

CPU-bound esetben a throughput 8 workernél éri el a maximumot (207 req/s), és e fölött monoton csökken: 32 workernél már csak 165 req/s mérhető. A p99/p50 arány ugyanebben a tartományban meredeken nő (7. ábra), ami arra utal, hogy a worker pool túlméretezésével megjelenő kontextusváltási overhead elsősorban a legrosszabb esetű válaszidőket rontja. Az, hogy az optimum a magok számánál ($c = 4$) nagyobb, több tényezővel magyarázható. Egyrészt a szerver kéréseket elfogadó szála maga is CPU-időt vesz el a kiosztott magokon, másrészt a task queue-ból való kivétel mutex-szel védett, ami rövid blokkolódási időket okoz a workerekben. Ezeket a pillanatok egy többlet worker hasznosítani tudja. Továbbá még CPU-bound taskok esetén is előfordulnak rövid várakozások memória késleltetés vagy rendszerhívások miatt, amelyek alatt a processzormag egyébként kihasználatlan maradna, és olyankor egy további worker képes dolgozni ezidő alatt.

IO-bound esetén a throughput görbe laposabb és zajosabb (5. ábra), ami a lemez változó elérési idejéből adódik. Az optimum szintén 8-12 worker körül van, és a throughput csökkenése az

optimum felett lényegesen lassabb, mint CPU-bound esetén. A tail ratio IO-bound esetén végig magasabb értéken stabilizálódik, ami azt tükrözi, hogy a fájlrendszer válaszidejének szórása önmagában is szélesíti a késleltetés eloszlását, függetlenül a worker pool méretétől.

Összességében a két task típus optimuma közelebb esett egymáshoz, mint ahogyan a hipotézisben vártam. A különbség a két task típus között nem az optimum helyében, hanem az optimum felett mutatott viselkedésben jelentkezik: CPU-bound esetén a degradáció meredek és a p99/p50 arány gyorsan nő, míg IO-bound esetén a rendszer toleránsabb a worker pool túlméretezésével szemben.

3.5. H3: Task queue méretének hatása

3.5.1. Hipotézis és felállítás

A worker pool előtti task queue-nak van egy optimális mérete, adott kliensszám és worker pool méret mellett. A túl nagy queue méret ún. "bufferbloat" jelenséget idéz elő, a túl kicsi hamar megtelik, és emiatt a szerver elutasítja a bejövő kéréseket. A worker pool előtti task queue-t, ami egy FIFO sor, tehát olyan sorrendben lépnek ki belőle az elemek, ahogyan bekerültek, azért hoztam létre, mert a bejövő kapcsolatok elfogadása és a kérések feldolgozása között van egy időbeli eltérés, mert különböző szálakon történnek. Emiatt biztosítanom kellett, hogy amíg nem veszi át őket egy worker feldolgozásra, legyen lehetőségem tárolni, és újabbakat elfogadni. A hipotézis motivációja egy nem dokumentált próbamérés, amit közvetlenül az implementáció után végeztem, és a task queue mérete és az átlagos latency között összefüggést mutatott. Ezért úgy döntöttem, hogy precízen is megvizsgálom ezt a kérdést, illetve a task queue mérete is a szerver egy szabadon hangolható paramétere, ezért a teljesség igényével ezt is vizsgálom.

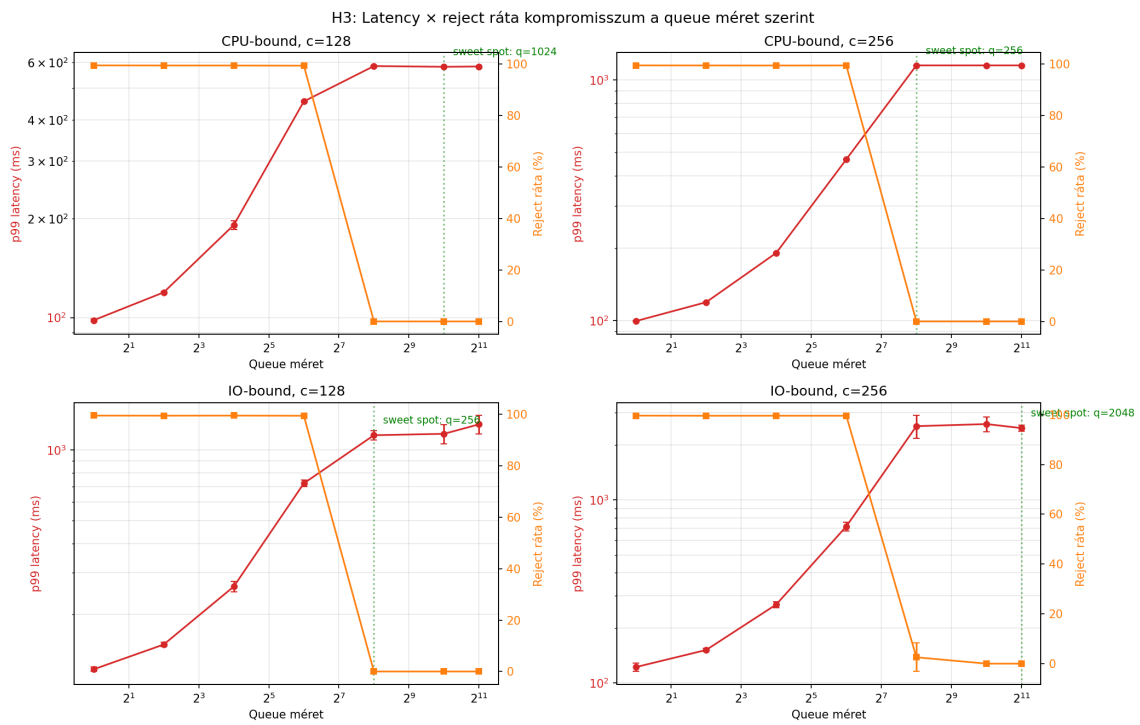
A mérési elrendezést úgy alakítottam ki, hogy a worker pool méretét kötöttre választottam meg, a H2 hipotézisben megtalált optimum értékére, ami 8 worker volt. Ez azért fontos, hogy a worker pool mérete ne legyen szűk keresztmetszet a mérés során, és így tisztán a task queue méretének hatása jelentkezzen. alapján 8-ra. A kliensszámot úgy választottam meg, hogy a worker pool telített legyen, és megjelenhessen a task queue hatása, azáltal, hogy egyre több várakozó kérés kerül bele. Két kliensszám, 128 és 256 mellett mértem. A task queue méretét rendre 1, 4, 16, 64, 256, 1024, 2048 értékeken iteráltam a mérés során, hogy széles tartományban vizsgálhassam a hatását. Vizsgáltam CPU és IO-bound taskokat is, hogy lássam, hogy a különböző jellegű feladatok esetén hogyan jelentkezik ez a hatás. A teljes mérést ötször futtattam, hogy átlagolni tudjam a kapott eredményeket minden beállítás mellett. Egy beállítás 60 másodpercig futott, hogy elegendő időt biztosítson a stabil méréshez. Köztük öt másodperc szünetet hagytam, hogy a rendszer vissza tudjon állni a kiindulási állapotba, és ne legyen hatással a következő mérésre. Előttük egy tíz másodperces "bemelegítési" futtatást csináltam, hogy még a mérés előtt hasonló állapotba kerülhessen a rendszer, mint futás közben. Ez alatt nem rögzítettem adatokat. Ezt követően két másodperc várakozási időt állítottam be, hogy a "bemelegítési" szakaszban futtatott terhelés közvetlenül ne befolyásolja az utána kezdődő tényleges mérést.

A nyolc logikai processzormag közül az első négyet, 0-3-ig a szerver számára osztottam ki, az

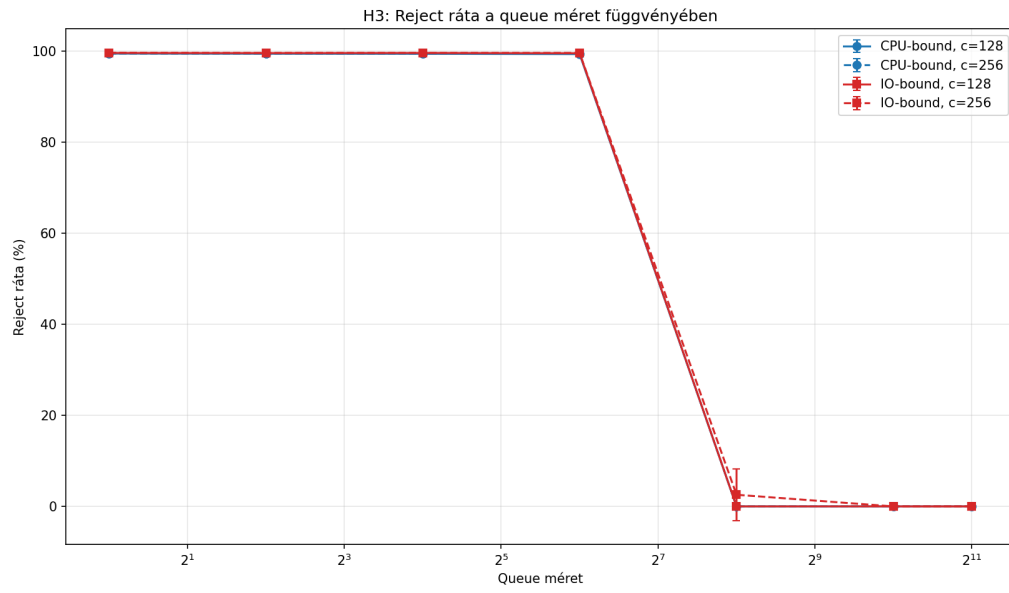
utolsó kettőt, 6-7 azonosítóval pedig a wrk mérőeszköz számára. Két processzormagot, 4-5 azonosítóval nem érintett a mérés, hogy ha az operációs rendszer valamilyen általam nem kontrollált háttér folyamatot futtatna, akkor az ne zavarja a mérésemet. Eközben mértem, hogy hány sikeres kérés valósul meg másodpercenként, és hogy a válaszidő milyen statisztikai jellemzőket mutat.

A hipotézis igazolt, ha létezik olyan queue méret, ahol a p99 latency mérhetően alacsonyabb ($>20\%$), mint a legnagyobb tesztelt méretnél, valamint ha a legkisebb tesztelt méreteknél (≤ 4) a reject ráta szignifikánsan ($>5\%$) nagyobb, mint az optimumnál, és a latency vs. queue méret görbe felismerhetően monoton növekvő egy bizonyos méret fölött, miközben a reject ráta közel nulla, ez igazolja az ún. bufferbloat-szerű viselkedést.

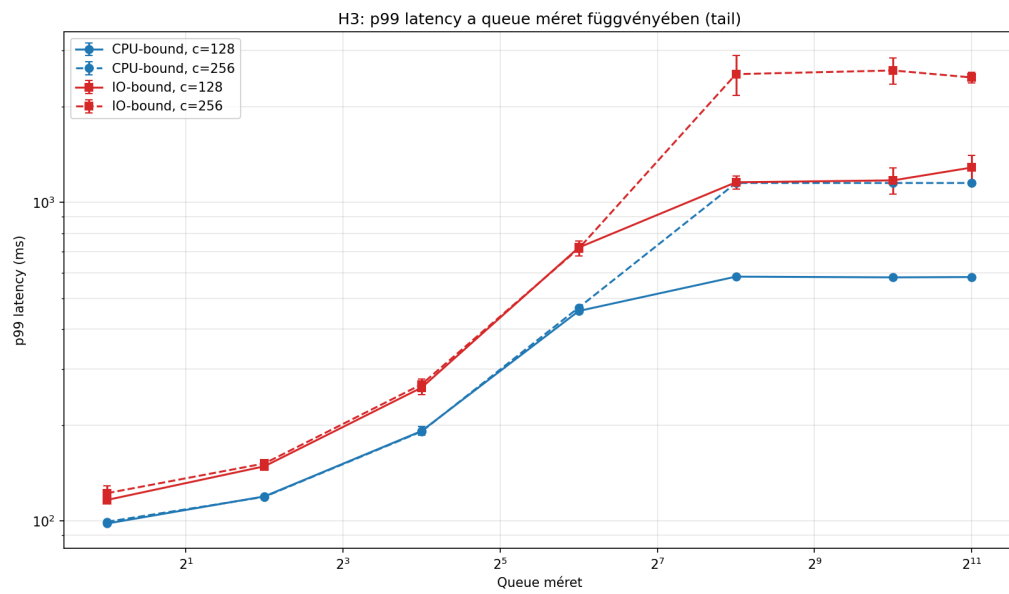
3.5.2. Eredmények



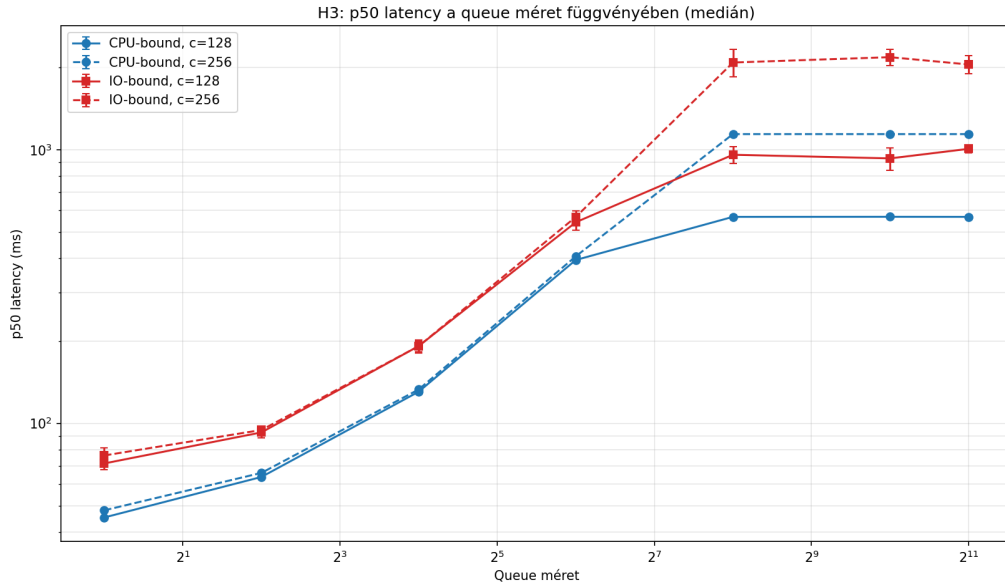
8. ábra. H3: Latency és reject ráta kompromisszum a queue méret függvényében.



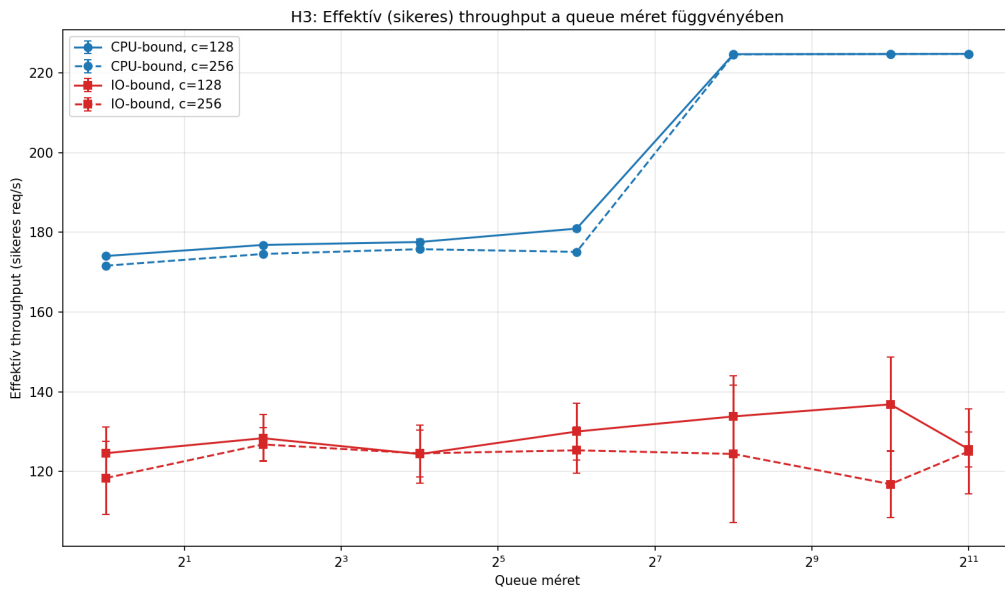
9. ábra. H3: Reject ráta lépcsőfüggvény jellege a queue méret függvényében.



10. ábra. H3: TODO cím



11. ábra. H3: TODO cím



12. ábra. H3: TODO cím

3.5.3. Diszkusszió

A mérési eredmények a hipotézist igazolják, és egy további, a hipotézisben nem szereplő jelenségre is rámutatnak. A visszautasított kérések aránya és a latency a queue méret függvényében egymással ellentétes irányban változik, és ez a kompromisszum mindkét task típusnál és mindkét kliensszámnál megfigyelhető (8. ábra).

A visszautasított kérések aránya lépcsőfüggvény jelleggel változik a queue méret függvényében (lásd 9. ábra): $q \leq 64$ -ig minden esetben közel 100%-os, majd egy küszöbértéknél hirtelen 0% közelébe esik. Ez a küszöb a kliensszámmal arányos: CPU-bound esetén $c = 128$ kliensnél

$q = 256$ -nál, $c = 256$ -nál szintén $q = 256$ -nál tűnik el a reject, míg IO-bound esetén valamivel nagyobb queue méret szükséges ugyanehhez. Ez azzal magyarázható, hogy a queue-nak legalább annyi helyet kell biztosítani, amennyi kérés egyidejűleg beérkezhet, mielőtt a workerek fel tudják dolgozni őket.

A mért latency a queue méret növelésével eleinte monoton nő, majd egy küszöb felett stabilizálódik (10. és 11. ábra). Ez a növekedés nem bufferbloat, hanem egy mérési sajátosság következménye: kis queue méreténél a kérések nagy része azonnal elutasításra kerül, és a wrk ezeket gyorsan visszakapja (TCP close), majd rögtön újat küld. A mért latency így csak azokra a kevés sikeres kérésre vonatkozik, amelyek épp belefértek a queue-ba, ezeknek viszonylag rövid volt a várakozási idejük, mert a queue sosem töltődött fel teljesen. Ahogy a queue mérete nő, egyre több kérés fér be, köztük olyanok is, amelyeknek tovább kell várniuk, hogy sorra kerülhessenek, ezért a mért átlagos latency nő. Amikor a queue elég nagy ahhoz, hogy ne keletkezzen reject, a sor hossza a rendszer terhelésétől függ, nem a queue korlátjától. Ettől a ponttól a queue mérete már nem befolyásolja érdemben a latencyt.

A nem várt megfigyelés az effektív throughput viselkedésében mutatkozik (lásd 12. ábra): CPU-bound esetben kis queue méreténél ($q \leq 64$) az effektív throughput 175 req/s, míg nagy queue méreténél 225 req/s. Ez azt jelzi, hogy a tömeges kérelmelutasítás maga is processzoridőt vesz el a szervertől. Minden elutasított kapcsolatot le kell zárni a TCP protokollnak megfelelően, ami csökkenti a hasznos feldolgozásra fordítható időt.

A sweet spot az az q érték, ahol a reject ráta már elhanyagolható, és a latency éppen stabilizálódott. Ez CPU-bound esetén $c = 128$ kliensnél $q = 1024$, $c = 256$ -nál $q = 256$, IO-bound esetén $c = 128$ -nál $q = 256$, $c = 256$ -nál $q = 2048$ körül azonosítható (8. ábra).

3.6. H4: Task típusok skálázódási profiljának összehasonlítása

3.6.1. Hipotézis és felállítása

A CPU-bound, IO-bound és network proxy taskok latency-throughput görbéi kvalitatívan különböznek a kliensszám függvényében: a knee point helye, a knee utáni meredekség, és a maximálisan elérhető throughput is task-típus-függő, és ezek a különbségek a feldolgozás CPU- és IO-időarányából magyarázhatók.

CPU-bound esetben a szűk keresztmetszet a magok száma. IO-bound esetben a workerek nagy része várakozik, így a CPU nem szűk keresztmetszet; a knee point később jelenik meg, és a throughput plafonját más limitálja (pl. fájlrendszer áteresztőképesség, vagy egy konstans feldolgozási overhead per request). A network proxy a függő rendszer (upstream) latencyjétől és kapacitásától függ, így viselkedése részben a saját szerverem paraméterein, részben az upstream szerveren múlik — ami egy harmadik, kvalitatívan eltérő profilt ad.

A mérési elrendezést úgy alakítottam ki, hogy a worker pool méretét a H2 hipotézis vizsgálata során meghatározott optimális értékére, 8 workerre állítottam be, hogy ez ne legyen szűk keresztmetszet a mérés során. A kliensszámot rendre a 1, 2, 4, 8, 16, 32, 64 tartományon iteráltam. Ez egy csökkentett tartomány, mert itt külső szervert elérő worker is futott, ami felé nagyobb

a késleltetés, és feltételeztem, hogy az egy címről történő elérések számát korlátozhatják a külső szerverek, ezért nem mentem el 128-ig vagy tovább. A task queue méretét nagyra, 2048-ra választottam, hogy ez ne legyen szűk keresztmetszet a mérés során. A teljes mérést ötször futtattam, hogy átlagolni tudjam a kapott eredményeket minden beállítás mellett. Egy beállítás 60 másodpercig futott, hogy elegendő időt biztosítson a stabil mérési eredmények eléréséhez. Közük öt másodperc szünetet hagytam, hogy a rendszer vissza tudjon állni a kiindulási állapotba, és ne legyen hatással a következő mérésre az előző.

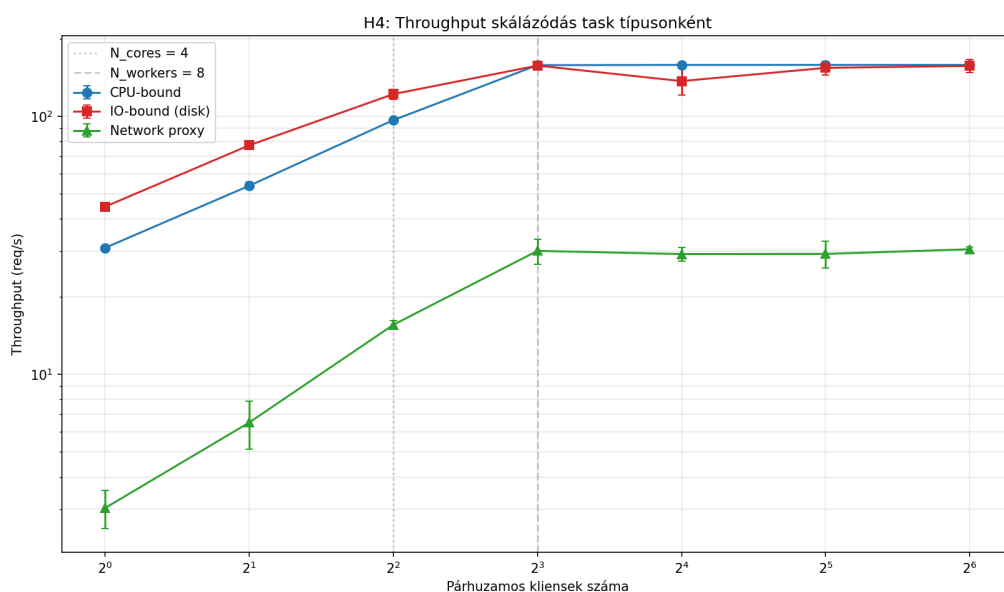
Előttük egy tíz másodperces "bemelegítési" futtatást csináltam, hogy még a mérés előtt hasonló állapotba kerülhessen a rendszer, mint futás közben. Ez alatt nem rögzítettem adatokat. Ezt követően két másodperc várakozási időt állítottam be, hogy a "bemelegítési" szakaszban futtatott terhelés közvetlenül ne befolyásolja az utána kezdődő tényleges mérést.

A nyolc logikai processzormag közül az első négyet, 0-3-ig a szerver számára osztottam ki, az utolsó kettőt, 6-7 azonosítóval pedig a wrk mérőeszköz számára. Két processzormagot, 4-5 azonosítóval nem érintett a mérésem, hogy ha az operációs rendszer valamilyen általam nem kontrollált háttér folyamatot futtatna, akkor az ne zavarja a mérésemet.

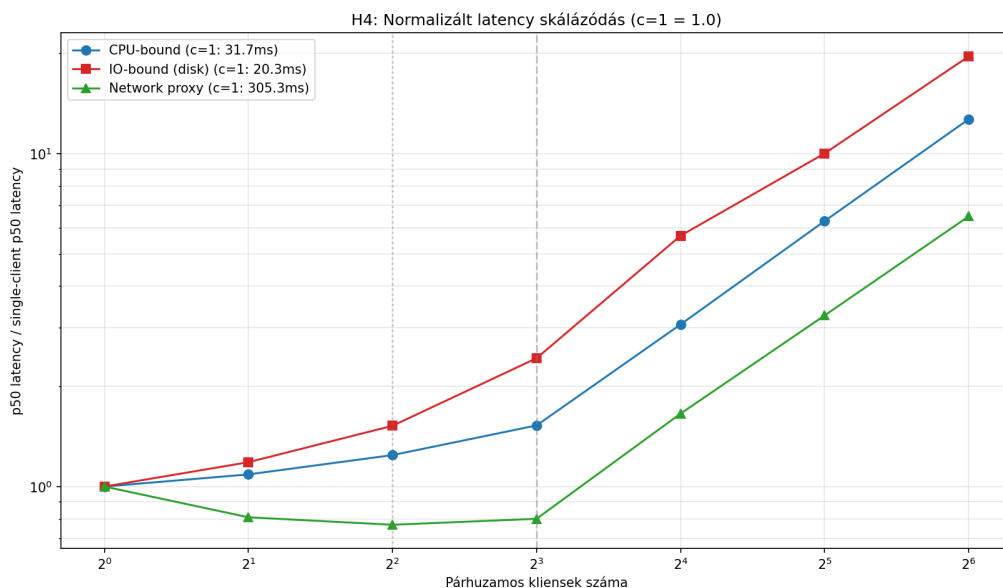
A network proxy taskok esetén a szerver egy valós, külső HTTP szerverhez kapcsolódott, a következő címen: "https://time.now/developer/api/timezone/Europe/London"

A hipotézis igazolt, ha a három task típus knee pointja között legalább 2×-es eltérés mérhető, a maximális stabil throughput a három task típusra különböző nagyságrendű vagy legalább 50%-ban eltérő, és a knee utáni latency-merevedés (pl. p95 növekedése egy logaritmikus klienslépésre vetítve) kvalitatívan különbözik — pl. a CPU-bound merevedékebb, mert ott a context switch overhead is bekapcsol, míg az IO-bound laposabban romlik.

3.6.2. Eredmények



13. ábra. H4: Throughput skálázódás task típusonként.



14. ábra. H4: Normalizált latency skálázódás (single-client érték = 1.0).

3.6.3. Diskusszió

3.7. Összegző megfigyelések

4. Összefoglalás

4.1. A hipotézisek áttekintése

Hipotézis	Státusz	Fő eredmény
H1	Igazolt	A knee point a worker pool méreténél van; A Little's Law teljesül
H2	Részlegesen igazolt	Az optimum nagyobb, CPU esetben nem magyszámnál van
H3	Igazolt	Új megfigyelés: a tömeges klienselutasítás csökkenti a hasznos throughputot
H4	Részlegesen igazolt	Új megfigyelés: a knee point közös (=worker pool mérete)

1. táblázat. A négy hipotézis eredményeinek áttekintése.

4.2. Továbbfejlesztési lehetőségek

A munkám során nem jutott időm arra, hogy megvizsgáljam, hogy milyen hatással van az IO-bound algoritmusokra az, hogy a processzor és feltételezéseim szerint a lemezmeghajtó is képes gyorsítótárazni, ún. "cache"-elni a feldolgozott adatokat. Ennek a hatása a kérések kiszolgálásának idejére jelentős lehet.

Találkoztam továbbá egy meglepő jelenséggel, amikor a worker pool méretének optimumát kerestem korlátozott mennyiségben rendelkezésre álló processzormag mellett. A htop rendszererőforrás-figyelő eszközzel meggyőződtem róla, hogy valóban kihasználják a CPU bound workerek a ren-

delkezésre álló processzoridőt minden magon. Ennek tudatában arra számítottam, hogy több worker létrehozásával, mint ahány processzormagot a szerver rendelkezésére bocsátok, csökkenni fog az egységnyi idő alatt kiszolgálható kérések száma, mert azonos erőforráson fognak kényte-lenné válni osztozni ezek a komponensek, és amíg az egyik példányuk processzoridőt kap egy adott magon, addig a többinek várakoznia kell, ezáltal meg fog nőni az idő, ami alatt ki tudja szolgálni a hozzá tartozó klienst. Ezzel szemben azt tapasztaltam, hogy a worker pool méretének optimuma négy processzormag mellett nyolc, ami ellent mond ennek a megközelítésnek. Ezt a jelenséget érdemes lenne még tovább vizsgálni, a pontos okának felderítése céljából.

Irodalomjegyzék

Hivatkozások

- [1] J. D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.
- [2] L. Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [3] Boost.Asio dokumentáció. https://www.boost.org/doc/libs/release/doc/html/boost_asio.html. (Letöltve: 2026).
- [4] libcurl — the multiprotocol file transfer library. <https://curl.se/libcurl/>. (Letöltve: 2026).
- [5] W. Glozer. wrk — a HTTP benchmarking tool. <https://github.com/wg/wrk>. (Letöltve: 2026).
- [6] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Pearson, 2. kiadás, 2020.